

ARCHITECTURE GUIDE

Micro Frontends for Backend Engineers

Libraries vs micro frontends, the four implementation patterns, authentication, hosting, converting an Angular library into a micro frontend, and a starter plan for learning Angular. Written for engineers and managers who know backend systems but are new to Angular.

October 2026 · Prepared for architecture, product and engineering discussions about "the app"

Contents

1. Why this guide
2. Angular in fifteen minutes, for a backend engineer
3. Two ways to split an app by domain: libraries or micro frontends
4. The four micro frontend patterns
5. The patterns side by side
6. Authentication: one login, owned by the shell
7. Hosting is not a pattern: S3, CDN and where files live
8. Converting an Angular library into a federated micro frontend
9. Applying it to the app
10. Leading the conversation
11. Getting started with Angular as a new manager
12. Glossary and sources

1. Why this guide

Several domain teams change one codebase and ship through one release. This guide explains the two ways to decouple them, and the technical patterns behind the micro frontend option, in backend terms.

"The app" is a large internal application used by several business domains. It began as a server-rendered Java application (JSP pages) and is being moved, page by page, to an Angular front end that calls REST APIs. Each modernized page sits behind a feature flag until it replaces the old one. This approach is often called the **strangler fig** pattern.

Different teams own different flows: an account opening team, a money movement team (contributions and grants), and other domain teams. Today they all change the same codebase and ship through one pipeline and one release, so:

- every release needs every team to merge, test and sign off together;
- one team's failing test or quality gate blocks every other team;
- a small change in one domain means regression testing the whole app;
- the slowest change sets the release pace for everyone.

Two options are on the table. Both give each domain team its own **vertical slice** of the app (screens, state and APIs), owned end to end. They differ in one thing only: whether a slice ships as a **library** that the app installs, or as a separately deployed **micro frontend**.

How to read this guide

Sections 2 and 3 give the background and the core decision. Sections 4 to 8 go deep on micro frontends: the patterns, login, hosting and the conversion steps. Sections 9 and 10 apply it to the app and help you lead the discussion. Section 11 is a starter plan for learning Angular.

2. Angular in fifteen minutes, for a backend engineer

An Angular app is a bundle of JavaScript that runs in the user's browser and calls your REST APIs. Most of its concepts map to things you already know from Spring.

How an Angular app differs from JSP

JSP (today's legacy pages)



Angular single-page app (the new UI)

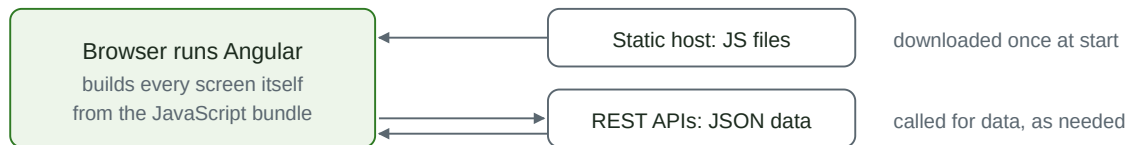


Figure 1. JSP renders pages on the server; an Angular app renders them in the browser and only fetches data.

Angular concepts mapped to Spring

Angular	Backend equivalent	What it does
Component	Controller plus its view template	One piece of a screen (a form, a table) with its own logic and HTML template
Service + dependency injection	@Service bean and @Autowired	Shared logic injected where needed; can be a singleton for the whole app
Router	@RequestMapping	Maps a URL such as /account-opening/step-2 to a component, inside the browser
Lazy loading	Loading a jar only when first needed	A section's code downloads only when the user first visits that route
HTTP interceptor	Servlet filter	Runs on every outgoing API call, for example to attach the login token
Route guard	A security filter on a URL pattern	Decides whether the user may open a screen (for user experience only; real security is in the backend)

Signals / RxJS Observables	Reactive streams, event listeners	How values change over time and how the screen updates when they do
TypeScript	Java-style static typing on JavaScript	The language Angular code is written in; compiled to JavaScript
ng build	<code>mvn package</code>	Compiles TypeScript into optimized JavaScript bundles
package.json / npm	<code>pom.xml</code> / Maven Central	Declares dependencies and versions; npm is the package registry
Angular library	A Maven dependency (jar)	Reusable code compiled into the consuming app at build time

Three facts that explain most of this guide

1. **Code runs on the user's machine.** A slow download or a bug is felt directly by users, and you cannot patch one user's browser. That is why bundle size and version mismatches matter more than on a server.
2. **The browser has security walls.** Code from one *origin* (scheme + domain + port) cannot freely read cookies or pages from another origin. Several patterns and login choices are shaped by this rule.
3. **Everything on one page shares one JavaScript runtime.** It is like many classes in one JVM: shared memory, shared global state, and one copy of each library unless you deliberately load two.

3. Two ways to split an app by domain

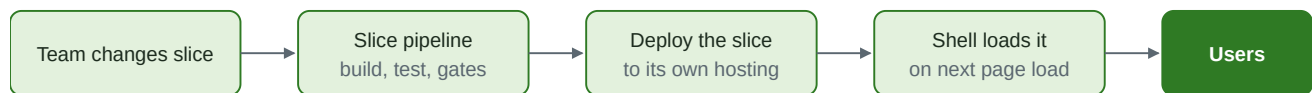
Both options give each domain team a vertical slice it owns end to end. The only difference is how a slice reaches users: installed into the app as a library, or deployed on its own as a micro frontend.

What both options share

- Each domain team owns its slice: screens and flows, state and API adapters, and ideally its domain APIs.
- One framework (Angular) and one design system across all domains.
- A shared foundation: login, shared contracts, monitoring.
- Feature flags and the strangler path for legacy pages.

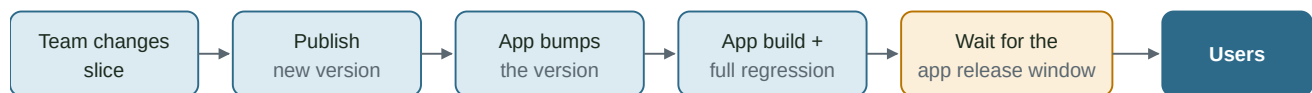
How a change reaches users

Micro frontend (deployable)



The owning team decides when. The app is not rebuilt.

Library (installed)



The app's release decides when. All Slices ship together.

Figure 2. The release path is the real difference between the two options.

Side-by-side comparison

Criterion	Micro frontend	Library	Edge
Time to first delivery	Slower: shell, hosting, pipelines first	Faster: works with today's app and pipeline	Library
Release independence	Each team deploys on its own	Teams publish; the app ships all slices together	Micro frontend
Lead time for one fix	Slice pipeline only	Slice pipeline + app integration + release window	Micro frontend
Setup effort	High: federation, runtime config, shell	Low: package registry and versioning	Library

Running cost	More pipelines, hosting, runtime config	One app pipeline plus library builds	Library
Version safety	Versions meet at runtime; mismatch possible	The compiler checks all slices together	Library
Impact of a bad release	One domain; rolled back alone	The whole app release; rolled back together	Micro frontend
Performance	Small runtime overhead; needs shared libraries	One optimized bundle	Library
Local development	Run the slice against a shell	Run the app as today	Library
Integration bottleneck	Only shell changes need the platform team	App team integrates every release	Micro frontend
Testing	Domain, contract and shell smoke tests	One integrated build, full regression	Even
User experience consistency	Same design system; needs governance	Same design system	Even

The library wins more rows; the micro frontend wins the rows tied to today's pain (release coupling).

Weigh the rows, don't count them.

Time to market

Library lead time = slice pipeline + app integration + **wait for the next app release**. Micro frontend lead time = slice pipeline. The gap is the app's release cadence. If the app can release daily with automated regression, the two are close. If releases are weekly or need manual regression, micro frontends are clearly faster. The library route is faster to start in either case, because it needs no new platform.

When each option fits

Choose micro frontends when

- Domain teams need different release schedules
- Whole-app regression is the bottleneck
- A defect in one domain must not block others
- Domains are loosely coupled
- There is capacity for a small platform group to own the shell
- The app keeps growing in teams and domains

Choose libraries when

- The app can release often, with automated regression
- A shared release is acceptable to teams
- Domains share a lot of state or navigation
- Little capacity for platform and runtime work
- Fastest start and lowest cost matter most
- Compile-time checks across slices are valued

A risk for both options

If domain APIs still deploy as one backend release, neither option gives teams real independence for changes that touch the API. Front-end independence then only helps UI-only changes.

What industry guidance says

- **Thoughtworks Technology Radar** rates micro frontends *Adopt* (since 2019) for independent team delivery, and rates "micro frontend anarchy" (mixing frameworks and approaches without standards) *Hold*.
- **The Angular team** (Angular blog, 2025) recommends first checking whether domain verticals in a monorepo, compiled together, are enough. Team autonomy in multi-team corporate settings is the main reason micro frontends get used.
- **Nx** recommends micro frontends when independent deployment is a hard requirement, and advises against them when teams already deploy together. It names mismatched shared library versions as a leading failure mode.

4. The four micro frontend patterns

Every pattern answers one question: where do separately deployed pieces get put together? In the same page, in a frame, or at the gateway.

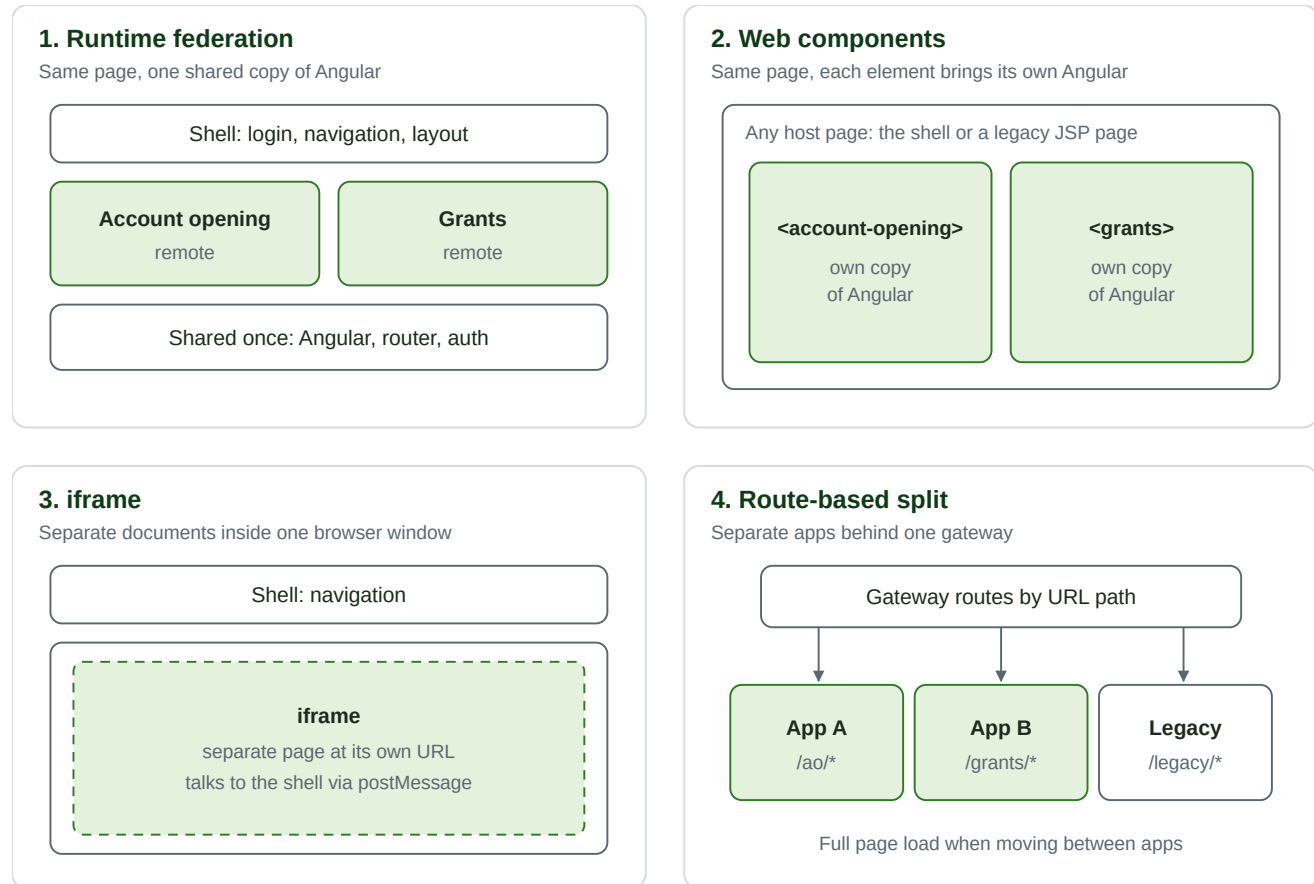


Figure 3. Green boxes are domain front ends deployed by their owning teams; plain boxes are the shell, gateway, host page or legacy app.

Pattern	Where the pieces come together	Backend analogy
1. Runtime federation	In the browser, inside one page, sharing one Angular	Plugins loaded at runtime that share a parent classloader (OSGi)
2. Web components	In the browser, inside one page, each with its own Angular	Self-contained fat jars with an embedded server
3. iframe	In the browser, in separate documents	Separate processes that talk by message passing
4. Route-based split	At the gateway, as separate pages	API gateway path routing to separate services

The closer together the pieces live, the smoother the experience and the more teams must coordinate on shared versions and rules. The further apart, the more isolated and independent, and the clunkier for users.

Pattern 1: Runtime federation

A thin Angular app, the **shell** (or host), loads each domain's code from a different URL while the app runs, and all of them share one copy of Angular. Each domain's separately deployed front end is a **remote**. To users it is one seamless app.

Backend analogy: an app server that loads separately deployed plugins at runtime, with Spring and logging loaded once by a parent classloader and shared by all plugins.

Tools: Module Federation came from webpack, Angular's older build tool. Native Federation does the same job on the newer esbuild-based Angular build and is the usual choice for recent Angular versions.

How a request flows

1. The user opens the app. The browser downloads the shell: header, navigation, login handling and router.
2. The shell reads a **manifest**, a small JSON file listing each remote and where its code lives. It is a service registry for UI code, with one per environment.
3. The user clicks *Open account*. The shell's router sees `/account-opening` belongs to a remote.
4. The shell downloads the remote's entry file, which declares what it needs (for example Angular 20). The shell already has Angular loaded, so it is shared, not downloaded again.
5. The account-opening screens render inside the shell's layout.

```
// Remote declares what it exposes (federation.config.js)
exposes: { './routes': './src/app/account-opening.routes.ts' }

// Shell loads it on a route
{ path: 'account-opening',
  loadChildren: () => loadRemoteModule('account-opening', './routes').then(m => m.ROUTES) }
```

Deploying: the team uploads new files to its hosting location. The next time a user opens that route, they get the new version. The shell is not rebuilt. Rollback means redeploying the previous files or pointing the manifest at them.

Login: everything runs in one page, so remotes use the shell's login, token and HTTP interceptor (see section 6).

How domains talk: through the URL (`/grants?donor=123`) and simple browser events. Avoid a shared state store across remotes; it is the front-end version of two services writing to one database table.

Main risk: version mismatch at runtime

If the shell runs Angular 20 and a remote was built for Angular 19, it can break in the user's browser, like a `NoSuchMethodError` between jars expecting different library versions. Fix it with governance: share only a few libraries, upgrade Angular majors together on a schedule, and check shared versions in CI.

Good: seamless app, smallest downloads, independent deploys, one login. **Watch:** version coordination, manifests and hosting per domain, testing remotes together. **Fits:** an all-Angular app with several teams.

Pattern 2: Web components (Angular Elements)

Each domain packages its UI as a **custom HTML tag**, such as `<account-opening-app donor-id="123">`. Any page can use it, whether an Angular shell or a server-rendered JSP page, because custom elements are a browser standard.

Backend analogy: a Spring Boot fat jar with an embedded Tomcat. It runs anywhere because it carries its own runtime, which is also why it is heavier.

1. The team builds its Angular code with Angular Elements, which wraps it as a standard custom element.
2. It publishes one JavaScript file to its hosting location.
3. A host page adds a `<script>` tag for that file and the custom tag where the UI should appear.
4. Data goes in as attributes or properties; results come out as browser events such as `accountCreated`.

Login: inside the shell, it can use the shell's auth service. On a legacy page, it relies on the existing session cookie, or the page passes a token in.

Why it matters: it is the only pattern that can put new Angular UI **inside an existing legacy page**, so part of a page can be modernized before the whole page moves.

Good: works in any host; versions can differ; clear contract. **Watch:** each element bundles its own Angular (bigger downloads, more memory); style clashes. **Fits:** new UI inside legacy pages, or widgets shared across frameworks.

Pattern 3: iframe

The shell shows a box (an iframe) that loads a completely separate page from its own URL: a browser window inside the page.

Backend analogy: two separate processes that share nothing and only exchange messages.

1. The shell renders its navigation plus `<iframe src="/legacy/accountOpening.jsp">`.
2. The browser loads that page fully separately: its own HTML, scripts, styles and memory.
3. The two talk only through `postMessage`, small JSON messages like a queue.

Login: on the same domain as the shell, cookies are shared and an existing session works. On a different domain, browsers increasingly block third-party cookies in iframes, so you need a token hand-off. Keep iframes same-origin, for example by serving legacy pages under `/legacy/*` through the gateway.

Common problems: the frame does not size to its content (scrollbars inside scrollbars); the back button and deep links get confusing; styling never quite matches; accessibility suffers; each frame loads everything separately.

Good: strongest isolation; any technology inside; quickest way to show a legacy page in a new shell.

Fits: a temporary bridge to legacy pages, rarely a long-term design.

Pattern 4: Route-based split

Each domain is a fully separate web app, and the gateway sends each URL path to the right one:

`/account-opening/*` to one app, `/grants/*` to another, `/legacy/*` to the existing Java app. Moving between domains is a normal full page load.

Backend analogy: exactly API gateway path routing for microservices, applied to whole web pages. This is the pattern you already know best.

Login: the session lives in a cookie on the shared domain, often set by a backend-for-frontend (BFF) or the gateway. Every app sits under the same domain, so the cookie travels with every page. It is the same mechanism the legacy pages use today.

Common problems: page reloads between domains; header and navigation duplicated in each app (share them through a library); a half-filled form is lost on navigation unless saved first.

Good: simplest; no runtime code sharing; fully independent apps; legacy pages slot in as routes. **Fits:** domains users rarely switch between mid-task, and routing to legacy pages during migration.

Other approaches you may hear about

- **single-spa:** an orchestrator that mounts apps built in different frameworks on one page. Adds little when every team uses Angular.
- **Server-side composition (SSI, ESI, edge includes):** a server or CDN stitches HTML fragments into one page. Common for content sites, rare for interactive business apps.

5. The patterns side by side

Federation gives the best experience with the most coordination; route-based and iframe give the most isolation with the clunkiest experience.

	1. Federation	2. Web components	3. iframe	4. Route-based
Backend analogy	Plugins sharing a parent classloader	Fat jar with embedded server	Separate processes	Gateway path routing
Where pieces combine	Browser, same page	Browser, same page	Browser, separate documents	Gateway, separate pages
User experience	Seamless	Seamless	Clunky (frames)	Page reloads
Copies of Angular	One, shared	One per domain	One per frame	One per app
Isolation	Low	Medium	Strong	Strong
Framework versions	Must match	Can differ	Can differ	Can differ
Login	Shell's auth service and interceptor	Shell's service or existing cookie	Same-origin cookie, else token hand-off	Shared domain cookie
Can show legacy JSP?	No	Yes, inside a page	Yes, wraps the page	Yes, as a route
Setup effort	Medium	Medium	Low	Low
Long-term fit	Main pattern for new Angular domains	Niche: UI inside legacy pages	Temporary bridge	Bridge, or loosely related domains

The usual real-world answer is a deliberate mix

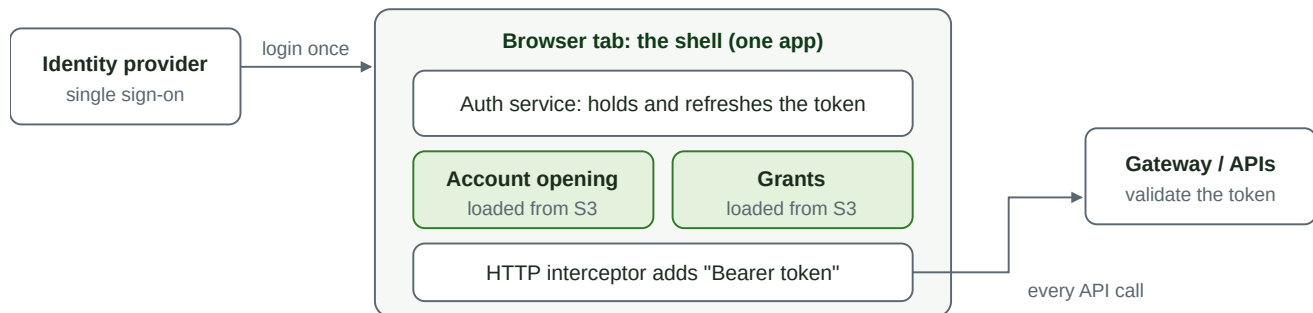
Federation for new Angular domains; route-based (or a same-origin iframe) to keep legacy pages reachable until they are retired; web components only where a legacy page must host new UI before it moves.

6. Authentication: one login, owned by the shell

With federation, micro frontends do not need their own login. The shell signs the user in once, and every micro frontend uses the shell's session and token.

Why federation needs no separate login

Federation loads the micro frontend's code **into the shell's own page**. Everything runs as one app in one browser tab, sharing one copy of Angular. So the micro frontend automatically uses the shell's login session, its `HttpClient`, and the auth interceptor that attaches the token to every API call.



1. The user signs in once through the shell.
 2. The shell keeps and refreshes the token.
 3. Remotes call their APIs through the shared interceptor, which adds the token.
 4. The backend checks every call.
- Remotes never handle login, refresh or logout.

Figure 4. In federation, remotes run inside the shell and reuse its login.

Three rules that make it work

1. **Share the HTTP library.** `@angular/common/http`, and your auth library if you have one, must be in the federation `shared` list so the remote uses the shell's copy.
2. **Do not re-create `HttpClient` in the remote.** If the remote's exposed routes call `provideHttpClient()` again, it gets its own `HttpClient` without the shell's interceptor, and API calls go out without the token. This is the most common bug.
3. **The backend still validates every call.** The shell decides who is logged in for the screens; the APIs and gateway decide what they may do. Route guards in the shell only hide screens; they are not security.

The standard login flow

1. The user opens the app; the shell finds no session and redirects to the identity provider.
2. After sign-in, the identity provider redirects back with a one-time code.
3. The shell exchanges the code for tokens using OpenID Connect (OIDC) with the authorization code flow and PKCE, the standard browser-safe approach.
4. The shell keeps the token in memory and refreshes it before it expires.

5. Every API call carries the token; logout clears the shell's and the identity provider's session.

A safer variant: backend-for-frontend (BFF)

A thin server holds the tokens and gives the browser only an `HttpOnly` session cookie, which JavaScript cannot read. API calls go through the BFF, which attaches the token on the server. This keeps tokens away from malicious scripts, and many security teams prefer it.

Login in each pattern

Pattern	How domains get the session	Note
1. Federation	Shell's auth service and interceptor	Simplest: everything is one page
2. Web components	Shell's service inside the shell; existing cookie or a passed-in token inside a legacy page	Agree who owns token refresh
3. iframe	Same-origin cookie; otherwise a token sent by <code>postMessage</code>	Keep frames same-origin
4. Route-based	Shared cookie on the common domain, set by a BFF or the gateway	Same mechanism as legacy pages

Carrying the session over to legacy pages

- **One domain, one session cookie (best):** serve legacy pages under the same domain through the gateway (for example `/legacy/`) and have both sides trust the same session.
- **Token exchange:** the gateway turns the new app's token into whatever the legacy app expects.
- **Silent SSO:** if the legacy app uses the same identity provider, it signs the user in invisibly.

7. Hosting is not a pattern: S3, CDN and where files live

S3 (often behind a CDN such as CloudFront) is where the files live. The four patterns describe how the pieces are combined. Any pattern can be hosted on S3.

What sits in S3	How the shell uses it	Pattern
Federation build output: <code>remoteEntry.json</code> + JS chunks	<code>loadRemoteModule('account-opening', './routes')</code>	1. Runtime federation
One bundled JS file built with Angular Elements	<code><script></code> tag + <code><account-opening-app></code>	2. Web components
A full app (<code>index.html</code> + JS)	Shown inside an <code><iframe></code>	3. iframe
A full app per domain	Gateway or CDN routes <code>/account-opening/*</code> to it	4. Route-based

Why you cannot upload the library's npm build directly

A natural question: "If the library is just JavaScript, why not put its files in S3 and load them from the Angular UI?" Three reasons:

1. **A library build is not finished code.** Angular libraries ship partially compiled. The consuming app's build finishes compiling them (the Angular "linker" step). A browser cannot run them as is. In Java terms: a jar compiled against `provided` dependencies, which a container is expected to complete.
2. **The browser cannot resolve `import '@angular/core'`.** At build time, the build tool finds Angular in `node_modules`. In the browser there is no `node_modules`. Federation solves this by pointing those imports at the copy of Angular the shell already loaded (using a browser feature called import maps). Without it, you would have to bundle a second Angular into the library's files, which is heavy and disconnects it from the shell's login and API handling.
3. **There is no contract.** The federation build adds `remoteEntry.json`, which lists what the remote exposes and which shared library versions it needs. A raw library build has nothing equivalent.

So the "wrapper app" is a build step, not a server

The remote wrapper project exists only to turn the library into browser-ready files. A CI job builds it and uploads the output to S3. Nothing runs on a server, and users never see it as a separate app.

```
Your library code
  |
  v
Wrapper project build (a CI job; no UI needed; nothing runs on a server)
  |
  v
Static files --upload--> S3: remoteEntry.json + a few .js files
                        |
                        v
Shell at runtime: "load account-opening from S3"
  -> uses its own Angular, login and interceptor -> renders the screens
```

Hosting checklist

- **Caching:** never cache `remoteEntry.json` (it points to the current version); cache hashed JS chunks for a long time.
- **Same origin or CORS:** serving remotes on the shell's domain (for example `/mfe/account-opening/` through the CDN) avoids cross-origin setup. A different domain needs CORS headers.
- **No secrets in files:** JavaScript files are public code, like the shell's own bundle. Data is protected at the API.
- **Manifest per environment:** dev, test and production point to different S3 or CDN paths.
- **Rollback:** redeploy previous files, or point the manifest at the previous version's path.

8. Converting an Angular library into a federated micro frontend

You do not convert the library itself. You keep it, add a thin remote wrapper build around it, and switch the UI app to load it at runtime instead of at build time.

Before: build time



After: runtime

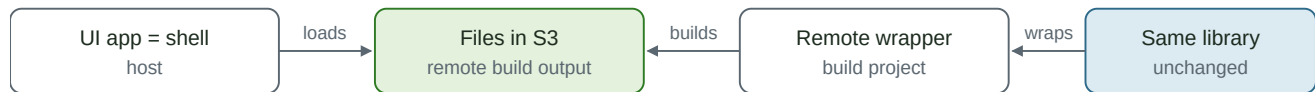


Figure 5. The library stays as it is; a wrapper build turns it into browser-ready files the shell loads at runtime.

Step 0: Make sure the library is ready

- **One route entry point.** The library exports its routes (for example `ACCOUNT_OPENING_ROUTES`) or one top-level standalone component.
- **Configuration through injection tokens.** API base URLs and flags are injected (for example `provideAccountOpening({ apiBase })`), not read from the host's environment files.
- **No hidden global needs.** List the global CSS, assets (images, translation files) and interceptors it expects.
- **No imports from the host app.** It depends only on Angular, the design system and shared packages.

Step 1: Create the remote wrapper project

```
ng generate application account-opening-mfe          # or a new repo for another team
npm install @your-org/account-opening@1.4.0        # the existing library
ng add @angular-architects/native-federation --project account-opening-mfe --type remote --port 4201
```

This creates `federation.config.js` and splits `main.ts` into `main.ts` + `bootstrap.ts`, so the browser can agree on shared library versions before Angular starts.

Step 2: Expose the library's routes

```
// account-opening-mfe/src/app/remote.routes.ts
import { Routes } from '@angular/router';
import { ACCOUNT_OPENING_ROUTES, provideAccountOpening } from '@your-org/account-opening';

export const ROUTES: Routes = [{
  path: '',
  providers: [provideAccountOpening({ apiBase: '/api/account-opening' })],
  children: ACCOUNT_OPENING_ROUTES,
}];

// federation.config.js
const { withNativeFederation, shareAll } = require('@angular-architects/native-federation/config');
module.exports = withNativeFederation({
  name: 'account-opening',
  exposes: { './routes': './projects/account-opening-mfe/src/app/remote.routes.ts' },
  shared: { ...shareAll({ singleton: true, strictVersion: true, requiredVersion: 'auto' }) },
},
});
```

`shared` is the key line: Angular, the router and `HttpClient` load once and are shared with the shell. Point the remote's own routes at `ROUTES` too, so the team can run it alone at `localhost:4201`.

Step 3: Turn the UI app into the shell (host)

```
ng add @angular-architects/native-federation --project main-app --type dynamic-host

// federation.manifest.json (one per environment)
{ "account-opening": "https://cdn.example.com/mfe/account-opening/remoteEntry.json" }
```

Step 4: Swap the import in the shell's routes

```
// Before: build time, from node_modules
{ path: 'account-opening',
  loadChildren: () => import('@your-org/account-opening').then(m =>
m.ACCOUNT_OPENING_ROUTES) }

// After: runtime, from the remote's URL
{ path: 'account-opening',
  loadChildren: () => loadRemoteModule('account-opening', './routes').then(m => m.ROUTES) }
```

Safe rollout tip

During the transition, keep the library installed in the shell and choose between the two lines with a feature flag. If the remote misbehaves in production, flip back to the built-in version without a deploy. Remove the library from the shell once the remote has proven itself.

Step 5: Handle what breaks most often

Concern	What to do
Login and API calls	Keep <code>provideHttpClient(withInterceptors([authInterceptor]))</code> in the shell and share <code>@angular/common/http</code> . Never call <code>provideHttpClient()</code> again in the remote.
Global styles	The shell does not load the remote's global <code>styles.css</code> . Move what the library needs into component styles or the shared design system.
Assets	Relative image and translation paths resolve against the shell's URL, not the remote's. Use absolute URLs or a configurable base URL. The classic "works locally, breaks deployed" bug.
Singleton services	Root services from a shared library live in the shell's injector. If the library is not shared and the shell also uses it, you get two copies and two copies of any state.
Remote fails to load	Catch errors from <code>loadRemoteModule</code> and route to a fallback or the legacy page, so users never see a blank screen.
Version match	Shell and remote must use the same Angular major version. Agree to upgrade together; <code>strictVersion</code> warns on mismatch.

Step 6: Build, deploy and test

- `ng build account-opening-mfe` produces static files; a CI job uploads them to S3 under the remote's path.
- Test at three levels: the remote alone; shell and remote together locally (ports 4200 and 4201); and a short end-to-end smoke test in the shell after every remote deploy.

What changes for each team

	Before	After
Library team	Publishes a package version	Still maintains the library; also owns the remote's pipeline and deploys to users directly
Shell team	Bumps the version, rebuilds, releases	Maintains the shell and the manifest; no rebuild for library changes
Users	Get changes on the app's release schedule	Get changes when the remote deploys

Command and option names follow Native Federation's documented setup and can change between major versions. Check the `@angular-architects/native-federation` README for the version that matches your Angular release.

9. Applying it to the app

If micro frontends are chosen, the app would use a deliberate mix of patterns during the migration, and could start from libraries without closing the door.

Need	Pattern	Why
New Angular domains (account opening, contributions, grants)	1. Federation	One seamless app, shared Angular, independent deploys
Legacy JSP pages not yet migrated	4. Route-based, or 3. same-origin iframe	Reachable without rework until retired
New UI inside a legacy page before it moves	2. Web components	The only pattern that lives inside a JSP page

How a click is routed during the migration

1. The shell holds a route map: each URL pattern points to a remote or to legacy, controlled by a feature flag.
2. A migrated route loads its remote inside the shell.
3. A legacy route goes through the gateway to the Java app (for example `/legacy/*` on the same domain) with the same session, so there is no second login.
4. When a remote reaches parity with its legacy page, the flag flips and the legacy route is retired.

A low-regret path

Build each domain slice as an Angular library with a clean route entry point and a public API. The app can install it now (library option), and a thin wrapper can later deploy the same library as a remote (micro frontend option). The choice can even differ per domain. Rules that keep a slice portable:

- No imports between slices; talk through routes and events.
- Get login and configuration through injected services.
- No global styles; each slice owns a route prefix.
- Depend only on the shared foundation.

Proposed proof of concept: build one slice, ship it both ways (4 weeks)

Week	Work
1	Build one account-opening step as an Angular library with a route entry point (using the shared account-opening component library).
2	Install it in the app and release one change end to end; record every step and wait.
3	Wrap the same library as a remote in a minimal shell; deploy it alone to S3 in a lower environment.
4	Fill in the scorecard and recommend a choice per domain.

Scorecard: lead time for a one-line fix; steps and hand-offs per release; setup effort (person-days); load time and bundle size; effect of a broken slice on the rest; rework to switch from library to micro frontend.

Proposed team: two engineers, an architect part time, one reviewer per domain team.

10. Leading the conversation

Open with the framing, not the tools: "Every option answers one question: where do separately deployed pieces get put together? The closer together, the smoother for users and the more coordination for teams."

Questions to ask the room

1. How often do users move between domains within one task? Often favors federation; rarely means route-based is enough.
2. Do we need new UI inside legacy pages before they move? If yes, web components have a role.
3. Can every team agree to upgrade Angular together? If not, federation becomes risky.
4. How does login work today, and is everything on one domain?
5. Can backend APIs deploy per domain? If not, front-end independence only helps UI-only changes.
6. How fast can the app release today? If daily with automated tests, libraries may be enough.

Pushback you are likely to hear

They say	You can answer
"Isn't this over-engineering?"	It can be. If the app can release often with automated regression, libraries may be enough. That is why the proof of concept measures both.
"Why not iframes? They're simple."	Good as a temporary bridge to legacy pages, but sizing, back-button and accessibility problems make them a poor long-term home.
"What if one domain breaks?"	With federation, the shell shows a fallback for that area, the rest keeps working, and the owning team rolls back alone. With a library, a defect blocks the whole app release.
"Won't it be slower?"	Federation shares Angular, so the overhead is small. Web components and iframes cost more. We will measure it in the proof of concept.
"Does each micro frontend need its own login?"	No. In federation the shell owns login and the remotes reuse its token and interceptor.
"Who owns the shell?"	A small platform group. The shell stays thin: login, navigation, layout, flags, monitoring, and no business logic.
"Can teams pick their own framework?"	No. One framework and one design system; mixing them is what Thoughtworks calls micro frontend anarchy.
"How do we test it all together?"	Domain tests per team, contract tests on the shell-to-remote interface, and an end-to-end smoke test in the shell on every deploy.

11. Getting started with Angular as a new manager

You do not need to become an Angular developer. You need enough to follow design discussions, judge risk, ask good questions and read a pull request. Four weeks of a few hours a week is enough to get there.

What to learn, and what to skip

Learn well enough to explain

- Components, templates and how a screen is built
- Services and dependency injection
- Routing, lazy loading and route guards
- HttpClient and interceptors (how API calls and login work)
- Signals: Angular's modern way to hold state and update the screen
- How the app is built, tested and deployed
- Angular's release and support policy

Skip for now

- Advanced RxJS operators
- CSS and layout details
- Animations
- Server-side rendering and hydration (rarely relevant for an internal app)
- Build-tool internals

A four-week plan

Week	Goal	Do this
1. Shape	Understand how an Angular app is put together	Install Node.js (LTS) and the Angular CLI. Run <code>ng new demo</code> and <code>ng serve</code> , then open the project and find <code>main.ts</code> , <code>app.config.ts</code> , <code>app.routes.ts</code> and one component. Work through the official tutorials at angular.dev (the "Essentials" section is about an hour). Keep the Spring mapping table from section 2 beside you.
2. Our codebase	See one feature end to end in the app	Book 90 minutes with a senior front-end engineer. Ask them to walk one feature from the route, to the component, to the service, to the API call and back. Ask where login, feature flags and shared state live, and how a build reaches production.

3. Hands on	Make one tiny change yourself	Change a label or add a read-only field on a screen, run the tests, open a pull request and watch it through the pipeline. Read one team pull request a day and ask one question on it.
4. Quality and risk	Know what "healthy" looks like	Look at test coverage, lint and Sonar results for TypeScript, bundle size, build time, flaky tests, and the Angular version and its end-of-support date. Agree with the team on two or three front-end health measures to track.

Questions to ask your team

- Which Angular version are we on, and when does its support end? Angular ships a major version about every six months, and each major is supported for about 18 months, so upgrades are a regular, plannable cost.
- Do we use standalone components (modern) or NgModules (older style), or a mix?
- How do we manage state: signals, services, or a library such as NgRx?
- How do login, the token and the HTTP interceptor work today?
- How long does a build and test cycle take, and which tests are flaky?
- What is our bundle size, and how fast does the first screen load?
- What are the top three things that slow you down in this codebase?
- Which parts would be hardest to split into domain slices, and why?

How to read an Angular pull request as a manager

- **Files ending in `.component.ts`, `.html`, `.scss`** change a screen. Ask: is there a test, and does it follow the design system?
- **Files ending in `.service.ts`** change logic or API calls. Ask: what happens on an error or a slow response?
- **`*.routes.ts`** changes navigation. Ask: is a new screen behind a feature flag, and does it need a guard?
- **`package.json`** changes dependencies. Ask: why this package, is it maintained, and does it affect bundle size or the Angular version?
- **`*.spec.ts`** are tests. A feature change with no spec change deserves a question.

12. Glossary and sources

Term	Meaning
SPA	Single-page application: the browser loads the app once and builds screens itself
Component	A piece of a screen with its template and logic
Service / DI	Shared logic provided through dependency injection
Signal	Angular's reactive value; the screen updates when it changes
RxJS / Observable	A library for streams of values over time, used by HttpClient and the router
Interceptor	Code that runs on every HTTP call, like a servlet filter
Route guard	Code that decides whether a screen may open (UX only, not security)
Standalone component	The modern way to write components without NgModules
Shell / host	The app that loads the micro frontends and owns login, navigation and layout
Remote	A domain's separately deployed front end
Manifest	Per-environment file listing where each remote lives
Shared singletons	Libraries loaded once and used by every piece (Angular, router, HTTP, auth)
Version skew	Pieces built against different library versions meeting at runtime
Native / Module Federation	Tools that load remotes at runtime (esbuild / webpack builds)
Angular Elements	The tool that packages Angular code as a web component
remoteEntry.json	A remote's contract: what it exposes and which shared versions it needs
Import map	A browser feature that tells the browser where to find a named module such as <code>@angular/core</code>
postMessage	The browser API for messages between a page and an iframe
Same origin	Same scheme, domain and port; the boundary for cookies and scripts
BFF	Backend-for-frontend: a thin server that holds the session and tokens for the UI
OIDC with PKCE	The standard browser-safe sign-in flow with an identity provider
Strangler fig	Replacing a legacy system piece by piece while both run side by side

Sources

- Thoughtworks Technology Radar, "Micro frontends": <https://www.thoughtworks.com/radar/techniques/micro-frontends>
- Thoughtworks Technology Radar, "Micro frontend anarchy": <https://www.thoughtworks.com/radar/techniques/micro-frontend-anarchy>
- Angular blog, "Micro Frontends with Angular and Native Federation" (Manfred Steyer, 2025): <https://blog.angular.dev/micro-frontends-with-angular-and-native-federation-7623cfc5f413>
- Nx documentation, "Micro frontend architecture": <https://nx.dev/concepts/module-federation/micro-frontend-architecture>
- Angular documentation and tutorials: <https://angular.dev>